

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Robotics and Automation Engineering

Positioning Systems Course

Unscented Kalman Filter for Simultaneous Localization and Mapping: Design and Evaluation

Prof.
Luigi D'Alfonso

Giuseppe Coppola
Mat. 263624

ACADEMIC YEAR 2025/26

Contents

Contents **1**

1 Introduction **3**

2 Unscented Kalman Filter **4**

2.1 Problem Formulation 5

2.1.1 Unscented Transform 5

2.1.2 UKF Algorithm 6

3 Simultaneous Localization and Mapping (SLAM) **9**

3.1 Formulation and Structure of the Problem 10

3.2 Probabilistic SLAM 11

3.3 Data Association 13

3.3.1 Batch Validation 14

3.3.2 Appearance Signatures 14

3.3.3 Multihypothesis Data Association 15

4 UKF-SLAM **16**

5 Design and Applications **18**

5.1 Simulation Enviroment 18

5.1.1 Robot Motion Model 19

5.1.2 Sensor Model 20

5.2 Simulation without Data Association 20

5.3 Simulation with Data Association 21

5.4 Results and Discussion 22

5.4.1 Best-Case Scenario 23

Pose Estimation Analysis 23

Landmark Estimation Analysis 24

Final Map Analysis 24
5.4.2 Middle-Case Scenario 25
Pose Estimation Analysis 25
Landmark Estimation Analysis 25
Final Map Analysis 26
5.4.3 Worst-Case Scenario 27
Pose Estimation Analysis 27
Landmark Estimation Analysis 27
Final Map Analysis 28

6 Conclusions **29**

Bibliography **31**

Chapter 1

Introduction

The Simultaneous Localization and Mapping (SLAM) problem consists of enabling a mobile robot to build a map of an unknown environment while simultaneously determining its own position within it. From a theoretical perspective, SLAM is formulated as a probabilistic estimation problem where the objective is to jointly estimate the robot's pose and the locations of environmental landmarks based on motion commands and sensor observations.

This report addresses the resolution of this problem by implementing the Unscented Kalman Filter (UKF). The UKF represents a derivative-free alternative to the Extended Kalman Filter (EKF), offering superior performance at a comparable computational cost. Unlike linearization-based methods, the UKF handles the nonlinear estimation problem by approximating the probability distribution through a carefully chosen set of sample points, known as "sigma points" (Unscented Transform), in order to more accurately capture the effects of nonlinearity.

The main objective of the project is to design and evaluate the performance of the UKF-SLAM algorithm within a two-dimensional simulation environment. In particular, the behavior of the filter is analyzed focusing on a critical aspect: Data Association, which is the process through which the robot must determine the correct correspondence between new sensor observations and the landmarks already present in its map.

Chapter 2

Unscented Kalman Filter

The **Unscented Kalman Filter** (UKF) represents a derivative-free alternative to the Extended Kalman Filter and provides superior performance at a comparable computational cost.

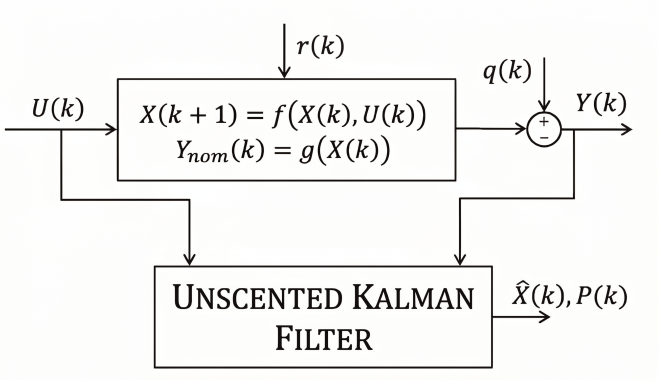


Figure 2.1: Framework with UKF

The main idea is to tackle the nonlinear estimation problem without relying on linearizations, instead using an **approximation of the nonlinear probability distribution**.

The UKF addresses the nonlinear estimation problem by representing the probability distribution of the state using a carefully chosen set of sample points, known as **sigma points**.

The propagation of these points through the nonlinear system functions, and the subsequent reconstruction of the mean and covariance from the transformed points, is known as the **Unscented Transform** (UT). This approach captures the effects of nonlinearity more accurately than linearization-based methods, such as those used in the Extended Kalman Filter.

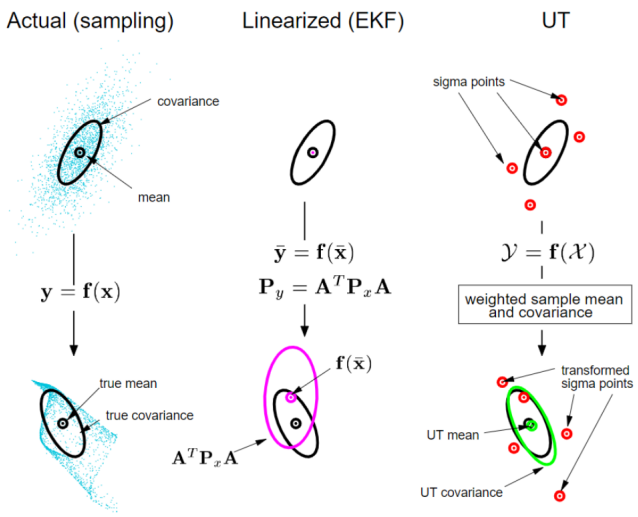


Figure 2.2: Linearization vs Unscented Transform

2.1 Problem Formulation

Consider the following discrete time non-linear system:

$$\begin{cases} x_{k+1} = f(x_k, u_k) + r_k \\ y_k = g(x_k, u_k) + q_k \end{cases} \tag{2.1}$$

with $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, $y \in \mathbb{R}^p$, $r \in \mathbb{R}^n$ and $q \in \mathbb{R}^p$.

By assuming that r_k and q_k are zero-mean Gaussian white uncorrelated noises with covariance matrices R and Q respectively.

2.1.1 Unscented Transform

Given a random variable x with:

$$E[x] = \bar{x}$$
$$\text{cov}[x] = E\left[(x - \bar{x})(x - \bar{x})^T\right] = P_x$$

and consider propagating it through a non-linear function $y = f(x)$.

The **objective** is to calculate the mean and covariance of y.

For this reason, a set of $2n + 1$ **sigma points** ξ is generated as:

$$\begin{aligned} \xi_0 &= \bar{x} \\ \xi_i &= \bar{x} + \left(\sqrt{cP_x}\right)_i \qquad \text{with } i = 1, \cdots, n \\ \xi_i &= \bar{x} - \left(\sqrt{cP_x}\right)_{i-n} \qquad \text{with } i = n+1, \cdots, 2n \end{aligned}$$

with $c = \alpha^2(n + \gamma)$ where:

- α determines the *spread* of the sigma points around $\bar{x}$
- γ is a secondary *scaling parameter*
- β is used to incorporate *prior knowledge* on the distribution

Theese sigma points are, then, propagated through the non linear function:

$$\psi = f(\xi)$$

and the mean and the covariance for ψ are approximated using a weighted sample mean and covariance of the posterior sigma points:

$$\hat{x} \approx \sum_{i=0}^{2n} w_i^{(m)} \psi_i$$
$$\hat{P}_x \approx \sum_{i=0}^{2n} w_i^{(c)} \left(\psi_i - \hat{x}\right) \left(\psi_i - \hat{x}\right)^T$$

with their relatives weights:

$$w_0^{(m)} = 1 - \frac{n}{c}$$
$$w_0^{(c)} = (2 - \alpha^2 + \beta) - \frac{n}{c}$$
$$w_i^{(m)} = w_i^{(c)} = \frac{1}{2c} \qquad \text{with} \quad i = 1, \cdots, 2n$$

2.1.2 UKF Algorithm

In the Unscented Kalman Filter, two main formulations can be adopted depending on how process and measurement noise are modeled.

In the **non-augmented** formulation, the noise is assumed to be additive in both the process and measurement models. In this case, the sigma points are generated only from the estimated state and its covariance matrix, while the effect of the noise is incorporated later through the addition of the process and measurement covariance matrices

In the **augmented** formulation, the process and/or measurement noise are included directly in an augmented state vector. Sigma points are therefore generated in the augmented state space, allowing the nonlinear transformation to propagate the uncertainty associated with both the state and the noise variables. This formulation is particularly useful when the noise enters the system in a non-additive manner.

Algorithm 1 Unscented Kalman Filter (UKF) Non-augmented

Input: $\hat{x}_{0|-1},\ P_{0|-1},\ \alpha,\ \beta,\ \gamma,\ c = \alpha^2(n+\gamma)$

for $k = 0 : N$ **do**

  Sigma points at $k|k-1$

    $\hat{\xi}^{(0)}_{k|k-1} = \hat{x}_{k|k-1}$

  for $i = 1 : n$ **do**

    $\hat{\xi}^{(i)}_{k|k-1} = \hat{x}_{k|k-1} + (\sqrt{cP_{k|k-1}})_i$

    $\hat{\xi}^{(i+n)}_{k|k-1} = \hat{x}_{k|k-1} - (\sqrt{cP_{k|k-1}})_i$

  end for

  Weights

    $w_0^{(m)} = 1 - \frac{n}{c}, \quad w_0^{(c)} = 2 - \alpha^2 + \beta - \frac{n}{c}$

    $w_i^{(m)} = w_i^{(c)} = \frac{1}{2c},\ i = 1, \ldots, 2n$

  Predicted measurements

  for $i = 0 : 2n$ **do**

    $\hat{z}^{(i)}_{k|k-1} = g(\hat{\xi}^{(i)}_{k|k-1}, u_k)$

  end for

    $\hat{y}_{k|k-1} = \sum_{i=0}^{2n} w_i^{(m)} \hat{z}^{(i)}_{k|k-1}$

    $P_y = Q + \sum_{i=0}^{2n} w_i^{(c)} (\hat{z}^{(i)}_{k|k-1} - \hat{y}_{k|k-1})(\hat{z}^{(i)}_{k|k-1} - \hat{y}_{k|k-1})^T$

    $P_{xy} = \frac{1}{2\alpha^2(n+\gamma)} \sum_{i=1}^{2n} (\hat{\xi}^{(i)}_{k|k-1} - \hat{x}_{k|k-1})(\hat{\xi}^{(i)}_{k|k-1} - \hat{x}_{k|k-1})^T$

  Kalman gain

    $L_k = P_{xy} P_y^{-1}$

  Update

    $\hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k(y_k - \hat{y}_{k|k-1})$

    $P_{k|k} = P_{k|k-1} - L_k P_y L_k^T$

  Sigma points at $k|k$

    $\hat{\xi}^{(0)}_{k|k} = \hat{x}_{k|k}$

  for $i = 1 : n$ **do**

    $\hat{\xi}^{(i)}_{k|k} = \hat{x}_{k|k} + (\sqrt{cP_{k|k}})_i$

    $\hat{\xi}^{(i+n)}_{k|k} = \hat{x}_{k|k} - (\sqrt{cP_{k|k}})_i$

  end for

  Prediction

  for $i = 0 : 2n$ **do**

    $\hat{\psi}^{(i)}_{k+1|k} = f(\hat{\xi}^{(i)}_{k|k}, u_k)$

  end for

    $\hat{x}_{k+1|k} = \sum_{i=0}^{2n} w_i^{(m)} \hat{\psi}^{(i)}_{k+1|k}$

    $P_{k+1|k} = R + \sum_{i=0}^{2n} w_i^{(c)} (\hat{\psi}^{(i)}_{k+1|k} - \hat{x}_{k+1|k})(\hat{\psi}^{(i)}_{k+1|k} - \hat{x}_{k+1|k})^T$

end for

Algorithm 2 Unscented Kalman Filter (UKF) Augmented

Input: $\hat{x}^{(a)}_{0|-1} = \begin{bmatrix} \hat{x}_{0|-1} & 0 & 0 \end{bmatrix}^T$, $P^{(a)}_{0|-1} = \begin{bmatrix} P_{0|-1} & 0 & 0 \\ 0 & R & 0 \\ 0 & 0 & Q \end{bmatrix}$, $\alpha,\ \beta,\ \gamma,\ c = \alpha^2(n+\gamma)$

for $k = 0 : N$ **do**

Sigma points at $k|k-1$

$\hat{\xi}^{(0)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1}$

for $i = 1 : n$ **do**

$\hat{\xi}^{(i)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1} + (\sqrt{cP^{(a)}_{k|k-1}})_i$

$\hat{\xi}^{(i+n)}_{k|k-1} = \hat{x}^{(a)}_{k|k-1} - (\sqrt{cP^{(a)}_{k|k-1}})_i$

end for

Weights

$w^{(m)}_0 = 1 - \frac{n}{c}, \quad w^{(c)}_0 = 2 - \alpha^2 + \beta - \frac{n}{c}$

$w^{(m)}_i = w^{(c)}_i = \frac{1}{2c},\ i = 1, \ldots, 2n$

Predicted measurements

for $i = 0 : 2n$ **do**

$\hat{z}^{(i)}_{k|k-1} = g(\hat{\xi}^{(i)}_{k|k-1}, u_k)$

end for

$\hat{y}^{(a)}_{k|k-1} = \sum_{i=0}^{2n} w^{(m)}_i \hat{z}^{(i)}_{k|k-1}$

$P^{(a)}_y = \sum_{i=0}^{2n} w^{(c)}_i (\hat{z}^{(i)}_{k|k-1} - \hat{y}^{(a)}_{k|k-1})(\hat{z}^{(i)}_{k|k-1} - \hat{y}^{(a)}_{k|k-1})^T$

$P^{(a)}_{xy} = \frac{1}{2\alpha^2(n+\gamma)} \sum_{i=1}^{2n} (\hat{\xi}^{(i)}_{k|k-1} - \hat{x}^{(a)}_{k|k-1})(\hat{\xi}^{(i)}_{k|k-1} - \hat{x}^{(a)}_{k|k-1})^T$

Kalman gain

$L_k = P^{(a)}_{xy}(P^{(a)}_y)^{-1}$

Update

$\hat{x}^{(a)}_{k|k} = \hat{x}^{(a)}_{k|k-1} + L_k(y^{(a)}_k - \hat{y}^{(a)}_{k|k-1})$

$P^{(a)}_{k|k} = P^{(a)}_{k|k-1} - L_k P^{(a)}_y L^T_k$

Sigma points at $k|k$

$\hat{\xi}^{(0)}_{k|k} = \hat{x}^{(a)}_{k|k}$

for $i = 1 : n$ **do**

$\hat{\xi}^{(i)}_{k|k} = \hat{x}^{(a)}_{k|k} + (\sqrt{cP^{(a)}_{k|k}})_i$

$\hat{\xi}^{(i+n)}_{k|k} = \hat{x}^{(a)}_{k|k} - (\sqrt{cP^{(a)}_{k|k}})_i$

end for

Prediction

for $i = 0 : 2n$ **do**

$\hat{\psi}^{(i)}_{k+1|k} = f(\hat{\xi}^{(i)}_{k|k}, u_k)$

end for

$\hat{x}^{(a)}_{k+1|k} = \sum_{i=0}^{2n} w^{(m)}_i \hat{\psi}^{(i)}_{k+1|k}$

$P^{(a)}_{k+1|k} = \sum_{i=0}^{2n} w^{(c)}_i (\hat{\psi}^{(i)}_{k+1|k} - \hat{x}^{(a)}_{k+1|k})(\hat{\psi}^{(i)}_{k+1|k} - \hat{x}^{(a)}_{k+1|k})^T$

end for

Chapter 3

Simultaneous Localization and Mapping (SLAM)

The **Simultaneous Localization and Mapping** (SLAM) problem consists of enabling a mobile robot to build a map of an unknown environment while simultaneously determining its own position within that environment. In other words, the robot must solve two fundamental problems in robotics at the same time: localization (estimating its position) and mapping (building a representation of the environment).
This problem has long been considered one of the main challenges in mobile robotics, since solving it allows a robot to operate autonomously in previously unknown environments. Over the past decades, SLAM has become one of the most important research fields in robotics, with applications ranging from indoor robots to autonomous vehicles, as well as underwater and aerial systems.
From a theoretical perspective, SLAM is formulated as a **probabilistic estimation problem**. The objective is to jointly estimate the robot's position and the locations of environmental landmarks based on:

- the robot's **control inputs** (motion)
- the **sensor observations** of landmarks

In this formulation, the goal is to compute the **joint probability** distribution of the robot state and the map of the environment, conditioned on the observations and control inputs collected over time. This problem is typically solved through a recursive process consisting of two main steps:

- **Prediction**: the new robot state is estimated based on the motion model.
- **Estimation**: the estimate is updated using sensor observations.

A fundamental characteristic of SLAM is that the estimated positions of landmarks become strongly correlated, since they all depend on the uncertainty of the robot's pose at the time the observations are made. As more measurements are collected over time, these correlations allow the accuracy of the map to progressively improve.

3.1 Formulation and Structure of the Problem

SLAM is a process by which a mobile robot can build a map of an environment and at the same time use this map to deduce its location.
In SLAM, both the trajectory of the robot and the location of all landmarks are estimated online without the need for any a priori knowledge of location.

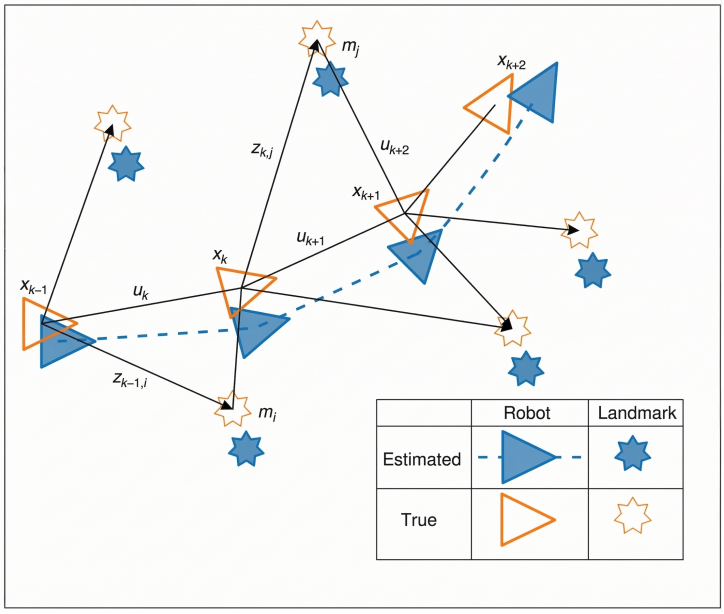


Figure 3.1: SLAM Overview

Consider a mobile robot moving through an environment taking relative observations of a number of unknown landmarks using a sensor located on the robot.
At a time instant k, it can be defined:

1. $\boldsymbol{x_k}$: state vector describing the location and orientation of the robot
2. $\boldsymbol{u_k}$: control vector, applied at time $k-1$ to drive the robot to a state x_k at time k
3. $\boldsymbol{m_i}$: vector describing the location of the i-th landmark whose true location is assumed time invariant
4. $\boldsymbol{z_{i,k}}$: observation taken from the robot of the location of the i-th landmark at time k

3.2 Probabilistic SLAM

In probabilistic form, SLAM problem requires that the probability distribution

$$p\left(x_k, m | z_{0:k}, u_{0:k}, x_0\right) \tag{3.1}$$

be computed for all times k.

This probability distribution describes the **joint posterior density** of the landmark locations and vehicle state given the recorded observations and control inputs up to and including time k together with the initial state of the robot.

In general, a recursive solution can be optained, using *Bayes Theorem* from

$$p(x_{k-1}, m | z_{0:k-1}, u_{0:k-2}, x_0) \tag{3.2}$$

and describing

$$p(x_k, m | z_{0:k}, u_{0:k-1}, x_0) = \mathcal{G}\left(p(x_{k-1}, m | z_{0:k-1}, u_{0:k-2}, x_0), u_{k-1}, z_k\right) \tag{3.3}$$

where $\mathcal{G}(\cdot)$ represents the recursive step.

This computation requires that a state transition model and an observation model are defined describing the effect of the control input and observation respectively.

The **observation model** describes the probability of making an observation z_k when the robot pose and landmark location are known. It is defined as

$$p(z_k | x_k, m) \tag{3.4}$$

It is reasonable to assume that once the robot location and map are defined, observations are conditionally independent given the map and the current robot state.

The **motion model** (transition model) can be described, for a given robot, in terms of probability distribution on state transition

$$p(x_k | x_{k-1}, u_{k-1}) \tag{3.5}$$

That is, the state transition is assumed to be a *Markov process* in which the next state x_k depends only on the immediately preceding state x_{k-1} and the applied control u_k and is independent both the observations and the map.

The SLAM algorithm is now implemented in a standard two-step recursive (sequential) prediction (time-update) correction (measurement-update) form:

- **Prediction - Time Update ($\boldsymbol{u_k}$ is used)**

$$p(x_k, m|z_{0:k-1}, u_{0:k-1}, x_0) = \int \boldsymbol{p(x_k|x_{k-1}, u_{k-1})} p(x_{k-1}, m|z_{0:k-1}, u_{0:k-2}, x_0)\, dx_{k-1} \tag{3.6}$$

- **Estimation - Measurement Update ($\boldsymbol{z_k}$ is used)**

$$p(x_k, m|z_{0:k}, u_{0:k-1}, x_0) = \frac{\boldsymbol{p(z_k|x_k, m)} p(x_k, m|z_{0:k-1}, u_{0:k-1}, x_0)}{p(z_k|z_{0:k-1}, u_{0:k-1})} \tag{3.7}$$

It can be seen that $p(m|x_{0:k}, z_{0:k}, u_{0:k-1})$ is a *mapping problem* and $p(x_k|z_{0:k}, u_{0:k-1}, m, x_0)$ is a *positioning problem.*

Referring to Figure (3.1), it can be seen that much of the error between estimated and true landmark locations is common between landmarks and is in fact due to a single source; errors in knowledge of where the robot is when landmark observations are made.
In turn, this implies that the errors in landmark location estimates are highly correlated. Practically, this means that the relative location between any two landmarks $(m_i - m_j)$ may be known with high accuracy, even when the absolute location of a landmark m_i is quite uncertain.
In probabilistic form, this means that the joint probability density for the pair of landmarks $p(m_i, m_j)$ is *highly peaked* even when the marginal densities $p(m_i)$ may be quite dispersed.
The most important insight in SLAM was to realize that the correlations between landmark estimates *increase monotonically* as more and more observations are made. Practically, this means that knowledge of the relative location of landmarks always improves and never diverges, regardless of robot motion. In probabilistic terms, this means that the joint probability density on all landmarks $p(m)$ becomes monotonically more peaked as more observations are made.

Convergence in SLAM occurs because the robot's observations act as *nearly independent* measurements of the relative distances between landmarks.
This process develops through three main mechanisms:

1. **Chain Propagation**: if the robot observes two landmarks (m_i and m_j) from position x_k, it creates a strong correlation between their positions. When it moves to x_{k+1} and only sees m_j again, the position update mathematically propagates to the unseen landmark m_i as well. Using the same measurement to update both further increases their correlation.

2. **Network Creation**: every newly observed landmark is immediately linked to the already known ones. All landmarks end up forming an interconnected network where every single observation cascades updates to the other nodes, constantly increasing their precision.

3. **Spring Analogy**: it can be visualized this network as a system of springs connecting all landmarks together. Each observation by the robot is like a pull that progressively stiffens the local springs and propagates the effect to nearby ones.

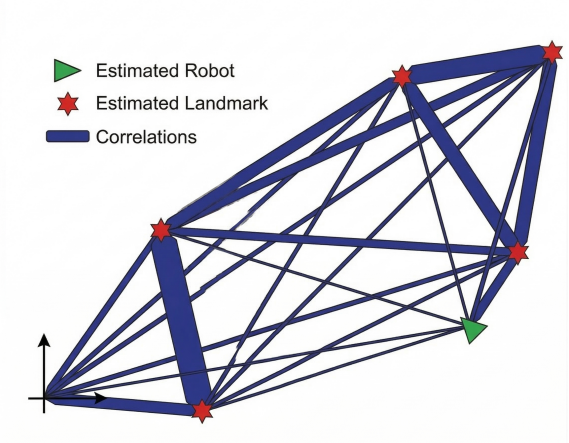


Figure 3.2: Spring Network Analogy

As the exploration continues, the "spring network" stiffens until it creates a highly accurate relative map. Once this limit is reached, the accuracy with which the robot knows its own position depends solely on the quality of its sensors and the precision of the map itself.

3.3 Data Association

Data association is a critical aspect in the practical implementation of SLAM. The problem can be described as follows.

The robot maintains a map containing a set of previously observed and stored landmarks. When the robot makes a new observation of the surrounding environment, it must determine how this observation relates to the existing map.
The key issue is determining which landmark in the map corresponds to the new observation. Alternatively, the observation may correspond to a completely new landmark that has never been seen before.

Before incorporating the new measurement into the map through the update step, the system must decide which landmark the observation should be associated with.

In many classical SLAM approaches, once a data association is made and the corresponding update step is performed, the decision cannot easily be reversed.
This creates a significant risk because a single incorrect association can cause the entire estimation of both the map and the robot's pose to diverge, potentially leading to a catastrophic failure of the algorithm. For this reason, many SLAM systems are considered fragile when they rely on perfectly correct data associations to function properly.

3.3.1 Batch Validation

Early SLAM approaches relied on **individual validation gating**, a technique originally developed for target tracking. This method evaluates each candidate association independently. If an observed landmark is sufficiently close to the predicted position of a landmark in the map (within a predefined gating threshold), the association is accepted. However, this approach becomes unreliable when the uncertainty in the robot's pose is high and generally fails in all but the simplest and most structured environments.

To address this limitation, **batch gating** methods consider multiple associations simultaneously by exploiting the *geometric consistency between landmarks*. Instead of validating each association independently, the set of associations must be mutually compatible. In other words, the relative geometry of the observed landmarks must match the relative geometry of the corresponding landmarks in the map.

Two common batch gating methods are **Joint Compatibility Branch and Bound** (JCBB) and **Combined Constraint Data Association** (CCDA). JCBB performs a tree search to find the largest set of associations that are jointly compatible. CCDA instead constructs a correspondence graph in which nodes represent candidate associations and edges connect geometrically compatible pairs. The data association problem is then solved by finding a clique in this graph, representing a set of mutually compatible associations.

These methods can be highly robust because they rely on the relative geometry of observations. When the geometric constraints are sufficiently strict, incorrect associations become rare and typically involve nearby landmarks, resulting in only minor inconsistencies.

3.3.2 Appearance Signatures

In addition to geometric constraints, data association in SLAM can also exploit appearance information provided by rich sensors such as cameras. Features such as shape, color, and texture, known as **appearance signatures**, can be used to match observations across

different datasets.

These features are useful both to predict potential associations, for example when the robot detects that it has returned to a previously visited place (loop closure), and to support geometric methods by providing additional discriminative information that improves the robustness of the data association process.

3.3.3 Multihypothesis Data Association

A particularly robust approach to the data association problem is **Multiple Hypothesis Tracking** (MHT), originally developed for target tracking in complex and cluttered environments.

The key idea is that when data association is ambiguous, the algorithm does not immediately commit to a single decision. Instead, it generates *multiple association hypotheses*, each producing a separate estimate of the map and the robot's pose. Over time, this process forms a tree of hypotheses. To keep the computation manageable, hypotheses with low probability are gradually pruned.

In the context of SLAM, this approach is particularly useful in large and complex environments and when the robot suspects that it has returned to a previously visited location. In such cases, the system can maintain two main hypotheses simultaneously: one where the location has already been visited and the map must be corrected, and another where the environment only appears similar but the robot is actually in a new place.

Chapter 4

UKF-SLAM

The basis is to describe the robot motion in the following form:

$$p(x_k|x_{k-1}, u_{k-1}) \iff x_k = f(x_{k-1}, u_{k-1}) + r_{k-1} \tag{4.1}$$

where $\boldsymbol{f(\cdot)}$ models the robot kinematics and $\boldsymbol{r_{k-1}}$ is additive, zero mean Gaussian motion noise with covariance $\boldsymbol{R}$.

The observation model is described by:

$$p(z_k|x_k, m) \iff z_k = g(x_k, m) + q_k \tag{4.2}$$

where $\boldsymbol{h(\cdot)}$ describes the geometry of the observation and $\boldsymbol{q_k}$ is additive, zero mean Gaussian observation noise with covariance $\boldsymbol{Q}$.

Moreover, the noises r_k and q_k are *uncorrelated*.

In a SLAM problem, the state does not have a fixed dimension but is variable; this is due to the fact that the robot, through its sensors, detects landmarks within the environment in different time istant.
It can be considered the following **augmented state**:

$$\zeta_k = \begin{bmatrix} x_k \\ m \end{bmatrix} \in \mathbb{R}^{n+L} \tag{4.3}$$

By applying the UKF method, and thus through the use of sigma points and the unscented transform (UT), the mean is calculated:

$$\hat{\zeta}_{k|k} = \begin{bmatrix} \hat{x}_{k|k} \\ \hat{m}_{k|k} \end{bmatrix} \tag{4.4}$$

Consequently, the augmented covariance matrix is given by:

$$\mathcal{P}_{k|k} = \begin{bmatrix} P_{xx} & P_{xm} \\ P_{xm}^T & P_{mm} \end{bmatrix}_{k|k} \in \mathbb{R}^{(n+L)\times(n+L)} \tag{4.5}$$

Given the mean and covariance, the model's state transition and observation equations are then derived.

$$\begin{cases} \zeta_{k+1} = \begin{bmatrix} x_{k+1} \\ m \end{bmatrix} = \begin{bmatrix} f(x_k, u_k) \\ m \end{bmatrix} + \begin{bmatrix} r_k \\ r_{m,k} \end{bmatrix} = \mathcal{F}(\zeta_k, u_k) + \tilde{r}_k \\ z_k = g(x_k, m) + q_k = g(\zeta_k) + q_k \end{cases} \tag{4.6}$$

where $\tilde{r}$ is a zero mean Gaussian noise with covariance $\mathcal{R} = \begin{bmatrix} R & 0_{n\times L} \\ 0_{L\times n} & R_m \end{bmatrix}$ and z_k with variable dimension d_k due to the number of measurements acquired at time step k.

Having also defined the equations governing the model, the **Estimation** and **Prediction** steps of the UKF algorithm can be readily applied:

- **Estimation Step - Measurements Update (z_k is used)**

$$\begin{aligned} L_k &= P_{xy} P_y^{-1} \\ \hat{\zeta}_{k|k} &= \hat{\zeta}_{k|k-1} + L_k \left(z_k - \hat{y}_{k|k-1} \right) \\ \mathcal{P}_{k|k} &= \mathcal{P}_{k|k-1} - L_k P_y L_k^T \end{aligned}$$

- **Prediction Step - Time Update (u_k is used)**

$$\begin{aligned} \hat{\zeta}_{k|k+1} &= \sum_{i=0}^{2n} \omega_i^{(m)} f(\epsilon_{k|k}^{(i)}, u_k) \\ \mathcal{P}_{k|k+1} &= \sum_{i=0}^{2n} \omega_i^{(c)} \left(f(\epsilon_{k|k}^{(i)}, u_k) - \hat{\zeta}_{k|k+1} \right) \left(f(\epsilon_{k|k}^{(i)}, u_k) - \hat{\zeta}_{k|k+1} \right)^T \end{aligned}$$

  where $\epsilon_{k|k}^{(i)}$ are the sigma points computed for $\hat{\zeta}_{k|k}$.

Chapter 5

Design and Applications

The goal of the simulations is to analyze the behavior of the UKF-SLAM algorithm under different assumptions regarding landmark association. In particular, two experimental scenarios have been considered. In the first scenario, the algorithm operates without performing data association, meaning that the correspondence between observed landmarks and map landmarks is assumed to be known a priori. This configuration allows the evaluation of the core estimation capabilities of the filter without the additional complexity introduced by the association process.

In the second scenario, data association is explicitly addressed. In this case, the algorithm must determine the correct correspondence between sensor observations and the landmarks already present in the map. This represents a more realistic situation, as in practical robotic applications the identity of observed landmarks is not known in advance.

The following sections describe the simulation environment, the robot and sensor models, the parameters used in the experiments, and the results obtained in both scenarios.

5.1 Simulation Enviroment

To evaluate the performance of the UKF-SLAM algorithm, a simulation environment was developed in Python. The environment models a mobile robot moving in a two–dimensional space populated with a set of static landmarks randomly distributed within a predefined area.

The robot moves according to a predefined trajectory and collects range and bearing measurements from nearby landmarks using a simulated sensor. These observations are affected by Gaussian noise in order to reproduce realistic sensing conditions.

The map is initially unknown to the robot. As the robot moves through the environment,

new landmarks are incrementally added to the state vector and estimated together with the robot pose.

5.1.1 Robot Motion Model

The robot is modeled as a differential-drive platform moving in a two-dimensional plane. Its pose is represented by the state vector:

$$\mathrm{x} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T \tag{5.1}$$

where x and y denote the cartesian position of the robot and θ the orientation.

The robot motion is described using an odometry-based motion model based on two rotations and one translation:

- $\boldsymbol{r_1}$: initial rotation
- $\boldsymbol{t}$: translation
- $\boldsymbol{r_2}$: final rotation

Therefore, the control input is given by:

$$u = \begin{bmatrix} r_1 & t & r_2 \end{bmatrix}^T \tag{5.2}$$

Then, the **state update** equation are defined as:

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + t\cos(\theta_k + r_1) \\ y_k + t\sin(\theta_k + r_1) \\ \theta_k + r_1 + r_2 \end{bmatrix} + r_k \iff \mathrm{x}_{k+1} = f(\mathrm{x}_k, u_k) + r_k \tag{5.3}$$

where $r_k \sim \mathcal{WN}(0, R)$ with

$$R = \begin{bmatrix} 0.05 & 0 & 0 \\ 0 & 0.05 & 0 \\ 0 & 0 & 0.005 \end{bmatrix}$$

Moreover, to simulate realistic conditions, Gaussian noise is added to the control inputs. The noisy control commands are used in the prediction step of the Unscented Kalman Filter.

5.1.2 Sensor Model

The robot is equipped with a range–bearing sensor capable of detecting nearby landmarks within a limited sensing region.

For each visible landmark, the sensor provides the following measurements:

1. **Range** (r): the Euclidean distance between the robot and the landmark
2. **Bearing** (φ): the relative angle between the robot orientation and the landmark direction

Given the robot pose (x, y, θ) and the landmark position (m_x, m_y), the **measurement model** is defined as:

$$\begin{bmatrix} \mathrm{r} \\ \varphi \end{bmatrix} = \begin{bmatrix} \sqrt{((m_x - x)^2 + (m_y - y)^2)} \\ \operatorname{atan2}(m_y - y, m_x - x) - \theta \end{bmatrix} + q_k \iff z_k = g(x_k) + q_k \tag{5.4}$$

where $q_k \sim \mathcal{WN}(0, Q)$ with

$$Q = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$$

Only landmarks located within the sensor field of view and within a maximum detection range are considered observable.

5.2 Simulation without Data Association

In this simulation scenario, the UKF-SLAM algorithm is executed under the simplifying assumption that the correspondence between observed landmarks and the landmarks already present in the map is known a priori. No explicit data association is performed. This allows the evaluation of the core estimation performance of the filter without introducing the additional complexity of landmark identification.

At each time step k, the UKF algorithm performs the following operations:

1. The robot state is propagated according to the motion model (5.3). The control input u_k is corrupted by Gaussian noise in order to simulate odometry uncertainty.
2. The Unscented Kalman Filter **prediction step** is performed.
3. Range and bearing measurements of all landmarks within the sensor range and field of view are simulated using the observation model defined in (5.4). Gaussian measurement noise is added to each observation.

4. For each received observation, the corresponding landmark in the state vector is identified using its known identifier.

5. If the observed landmark has already been included in the state vector, the UKF **correction step** is performed.

6. If the observed landmark has not been previously detected, its position is initialized using the inverse observation model and the state vector is augmented by adding the new landmark coordinates together with the corresponding covariance terms.

This configuration allows analyzing the evolution of the robot pose estimation and the landmark map without the confounding effects of incorrect data associations.

5.3 Simulation with Data Association

In this simulation scenario, the UKF-SLAM algorithm is executed with explicit data association. In this case, the algorithm must determine the correct correspondence between sensor observations and the landmarks already present in the map. This represents a more realistic situation, as in practical robotic applications the identity of observed landmarks is not known in advance.

At each time step k, the UKF algorithm performs the following operations:

1. The robot state is propagated according to the motion model (5.3). The control input u_k is corrupted by Gaussian noise in order to simulate odometry uncertainty.

2. The Unscented Kalman Filter **prediction step** is performed.

3. Range and bearing measurements of all landmarks within the sensor range and field of view are simulated using the observation model defined in (5.4). Gaussian measurement noise is added to each observation.

4. **First Step Initialization**: if the map is currently empty, all incoming observations are immediately classified as new landmarks. The algorithm augments the state vector and covariance matrix by adding these landmarks to the map.

5. **Data Association via Mahalanobis Distance**: the map already contains landmarks, the algorithm must associate new observations with the existing map. It evaluates each new observation against all mapped landmarks that have not yet been matched during the current time step. For each candidate pair, the system

computes the **Mahalanobis distance** (D^2). This distance is calculated using the innovation (the difference between the actual observation and the predicted measurement) and the innovation covariance matrix:

$$D^2 = \text{innovation}^T S^{-1} \text{innovation} \tag{5.5}$$

6. **Gating and Matching**: an observation is associated with the map landmark that yields the lowest Mahalanobis distance. This association is only accepted if the minimum distance is strictly below a predefined validation gate threshold (γ):

$$D^2 < \gamma \tag{5.6}$$

where $\gamma = 5.991$ which corresponds to the 95% confidence region for a chi-squared distribution with 2 degrees of freedom.

7. **Measurement Update**: for each observation successfully associated with an existing landmark, the algorithm performs the standard UKF correction step. It calculates the Kalman gain and updates the predicted state vector and covariance matrix to incorporate the geometric constraints of the measurement. Once updated, the specific landmark is marked as processed so it cannot be matched with other observations in the same step.

8. **Map Augmentation**: observations that fail to associate with any existing landmark (because their lowest Mahalanobis distance exceeds the 5.991 threshold) are categorized as newly discovered landmarks. The algorithm proceeds to initialize these new observations into the map by augmenting both the state vector and the covariance matrix.

5.4 Results and Discussion

This section presents and analyzes the results obtained from the UKF-SLAM simulations. The performance of the algorithm is evaluated under different environmental and noise conditions, as well as under different assumptions regarding data association.

In order to provide a comprehensive assessment, three distinct simulation scenarios have been considered:

- **Best-case scenario**: characterized by a high density of landmarks and low measurement noise, providing favorable conditions for both localization and mapping.

- **Middle-case scenario**: represents a moderate environment in terms of landmark density and sensor noise, offering a balanced and realistic operating condition.
- **Worst-case scenario**: defined by a sparse landmark configuration and higher noise levels, leading to a more challenging SLAM problem and increased uncertainty in both pose and map estimation.

5.4.1 Best-Case Scenario

The best-case scenario is designed to evaluate the UKF-SLAM performance under near-ideal conditions, characterized by a dense distribution of landmarks (35 landmarks) and low measurement noise. These optimal conditions provide a strong baseline to verify the robustness of the Data Association (DA) algorithm based on the Mahalanobis distance, comparing its performance directly against the idealized system with known correspondences (Perfect IDs).

Pose Estimation Analysis

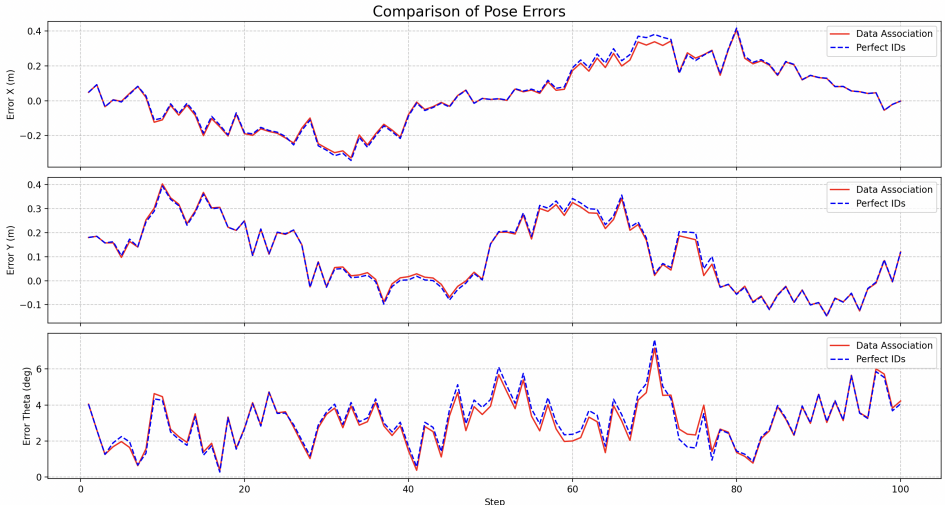


Figure 5.1: Pose estimation error (Perfect IDs vs DA) in the best-case scenario.

The near-perfect overlap of the two curves demonstrates that the DA algorithm effectively prevents misassociations, avoiding trajectory divergence. Translational errors are strictly bounded within ± 0.4 meters, and orientation errors peak below $8°$. Minor drifts occur when navigating areas with fewer visible landmarks, but the filter shows rapid convergence upon re-observing mapped features.

Landmark Estimation Analysis

A simultaneous error peak occurs around step 50, corresponding to a planned reversal of the robot's angular velocity (ω), which injects kinematic uncertainty into the prediction step. However, as stable measurements are reacquired, the errors rapidly decrease. The identical transient and convergence patterns in both subplots confirm that the DA logic operates as reliably as the Perfect IDs system.

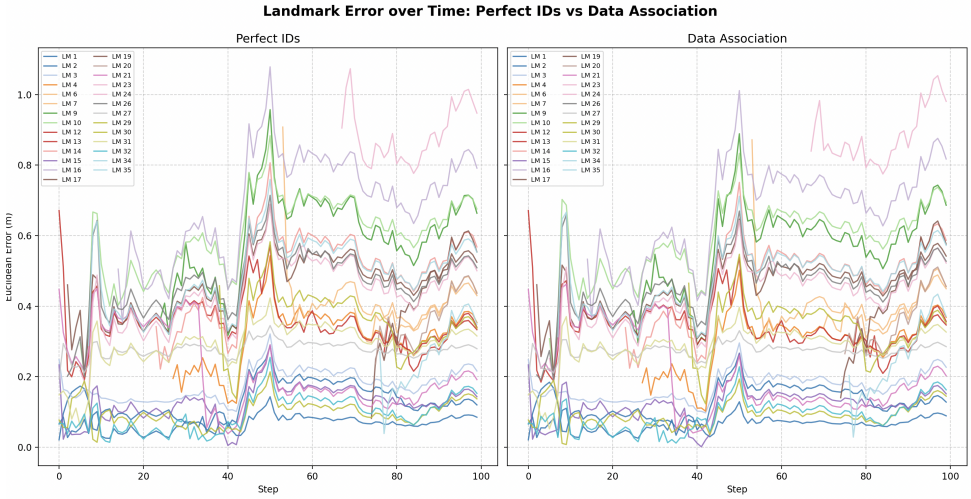


Figure 5.2: Landmark estimation error (Perfect IDs vs DA) in the best-case scenario.

Final Map Analysis

To provide a comprehensive visual summary of the simulation, Figure 5.3 illustrates the final state of the environment. This representation encompasses the true and estimated trajectories of the robot alongside the mapped landmarks, offering a direct qualitative comparison between the Perfect IDs and Data Association approaches.

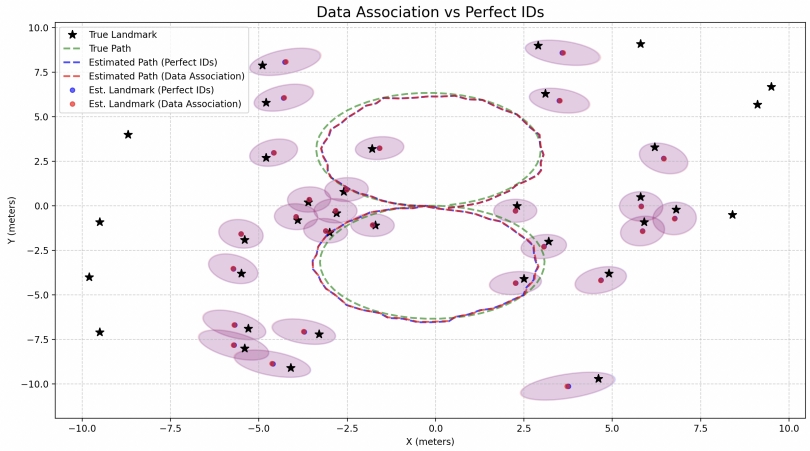


Figure 5.3: Final map of the environment in the best-case scenario.

5.4.2 Middle-Case Scenario

The middle-case scenario represents a more balanced and realistic operating condition, characterized by a moderate landmark density ($N = 16$) and increased measurement noise. To provide a visual summary of this configuration.

Pose Estimation Analysis

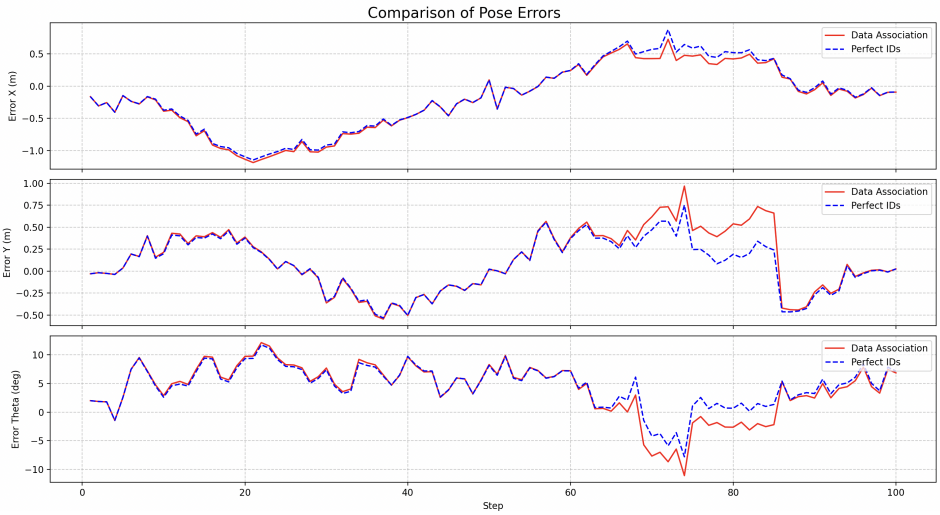


Figure 5.4: Pose estimation error (Perfect IDs vs DA) in the middle-case scenario.

Due to the higher noise and sparser map, the overall error magnitudes are larger than in the best-case scenario, with translational errors reaching up to ± 1.0 meters and orientation errors peaking around 11°. A critical observation occurs between steps 65 and 85: the Data Association (DA) trajectory temporarily diverges from the Perfect IDs baseline, particularly visible in the y-axis error. This deviation is caused by the increased measurement noise and the sparsity of features, which make the Mahalanobis gating more challenging. However, the filter demonstrates strong resilience by successfully correcting the drift and converging back to the baseline around step 85, effectively performing a loop closure.

Landmark Estimation Analysis

The Euclidean errors are overall higher, with peaks approaching 1.75 meters for certain features (e.g., LM 16). Despite the noisier environment, both the Perfect IDs and DA approaches exhibit similar behavioral trends. Notably, a sharp reduction in the estimation error across multiple landmarks occurs just after step 80. This event directly correlates with the pose correction phase observed in the trajectory error: once the robot re-observes a reliable set of previously mapped landmarks, the DA algorithm successfully matches the

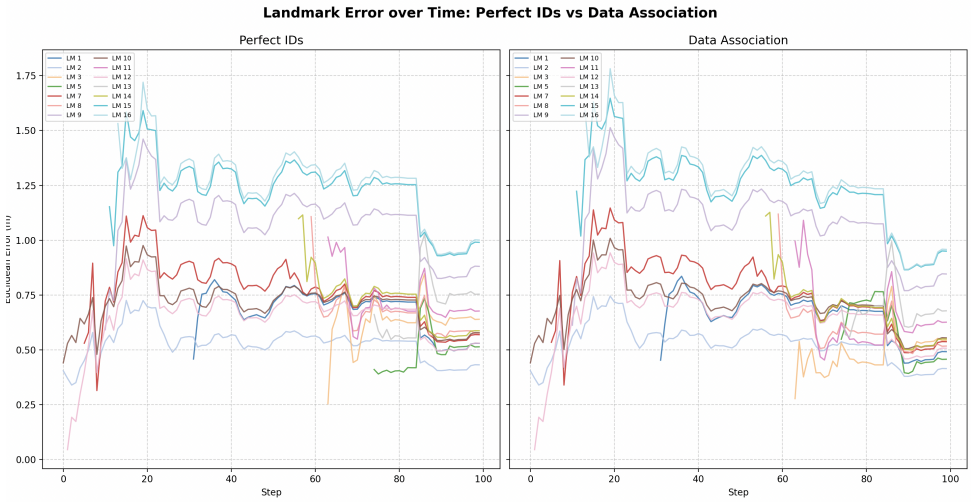


Figure 5.5: Landmark estimation error (Perfect IDs vs DA) in the middle-case scenario.

observations, reducing the accumulated uncertainty and restoring the spatial consistency of the entire SLAM system.

Final Map Analysis

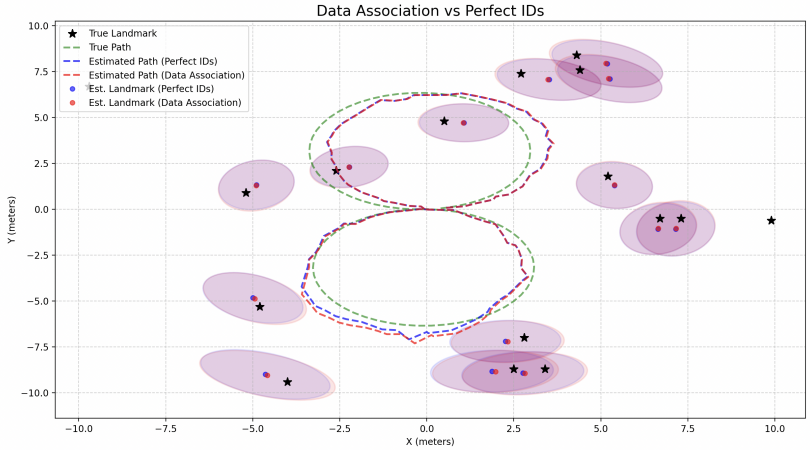


Figure 5.6: Final map of the environment in the middle-case scenario.

The visual contrast with the best-case scenario is also evident in the landmark covariance ellipses: they are significantly larger and more elongated, directly reflecting the higher uncertainty managed by the UKF. However, despite this increased ambiguity, the Data Association path sharply converges back to the ground truth upon re-entering previously mapped areas. This successful loop closure demonstrates that, although more challenged than in ideal conditions, the Mahalanobis distance gating remains sufficiently robust to recover from temporary drift and maintain global map consistency.

5.4.3 Worst-Case Scenario

The worst-case scenario evaluates the algorithm under extreme conditions: a highly sparse environment ($N = 4$) coupled with the highest level of measurement noise.

Pose Estimation Analysis

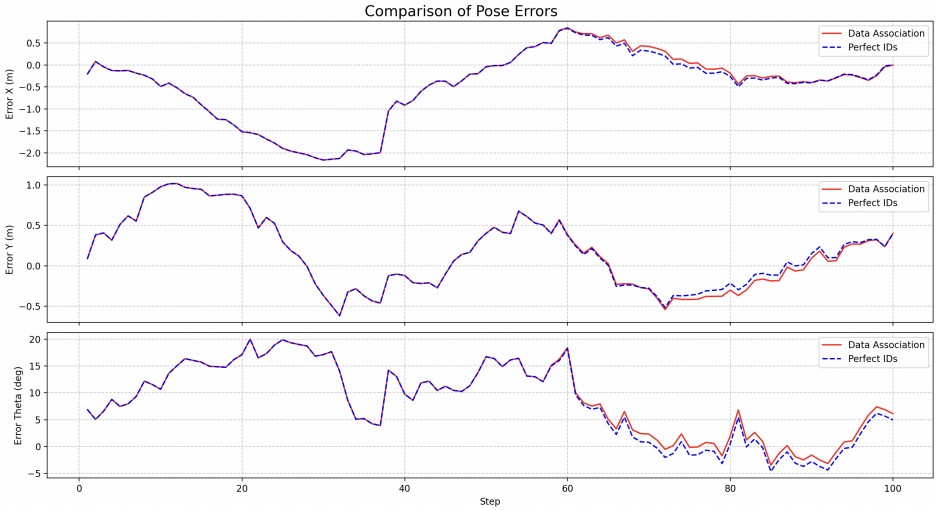


Figure 5.7: Pose estimation error (Perfect IDs vs DA) in the worst-case scenario.

he pose errors are substantially larger than in all previous scenarios, with translational errors reaching up to -2.0 meters on the X-axis and orientation errors peaking at 20°. However, a crucial insight emerges from this comparison: the Data Association curve (red) closely tracks the Perfect IDs baseline (blue) throughout the entire simulation, with only minor deviations in the final steps. This indicates that the severe loss of accuracy is caused by the inherent lack of geometric constraints (sparsity and noise), rather than a failure of the Mahalanobis association logic, which continues to correctly match the few available observations.

Landmark Estimation Analysis

Initial positioning errors are extreme, with LM 1 starting near 4.0 meters of Euclidean error. Due to the sparsity, landmarks remain unobserved for long periods, causing the errors to plateau. Nevertheless, sharp, step-like reductions in error occur (e.g., LM 2 around step 60) when the robot finally re-observes a feature and performs a filter update. The nearly identical step-responses in both the Perfect IDs and DA subplots reaffirm that, even in a highly degraded and ambiguous environment, the data association algorithm performs optimally without introducing false positives.

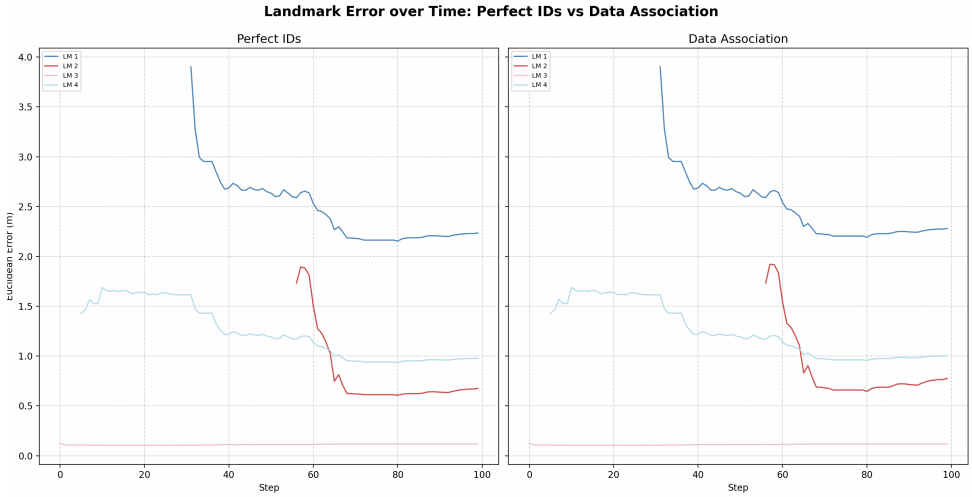


Figure 5.8: Landmark estimation error (Perfect IDs vs DA) in the worst-case scenario.

Final Map Analysis

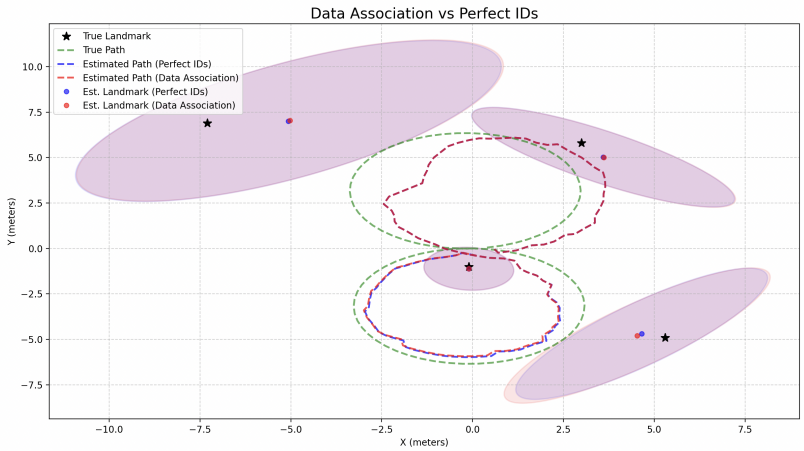


Figure 5.9: Final map of the environment in the worst-case scenario.

The robot relies heavily on odometric integration (dead reckoning), causing both the Perfect IDs and Data Association trajectories to visibly drift from the ground truth. However, the critical finding is the near-perfect overlap between the two estimated paths. This demonstrates that the Mahalanobis distance gating ($\gamma = 5.991$) remains robust and successfully avoids misassociations. The observed geometric drift is entirely attributable to the lack of environmental constraints, proving that the data association logic remains reliable even under extreme uncertainty.

Chapter 6

Conclusions

The simulations conducted allowed for the analysis of the UKF-SLAM algorithm's behavior under various environmental and noise conditions, comparing the performance of an ideal system with known landmark identities ("Perfect IDs") against a more realistic system equipped with Data Association (DA) logic based on the Mahalanobis distance.

The obtained results are summarized in the following three simulation scenarios:

- **Best-Case Scenario**: In an environment with a high density of landmarks and low sensor noise, the data association algorithm effectively prevents misassociations.

  The error curves of the DA system and the ideal system overlap almost perfectly, keeping translational errors within ± 0.4 meters and demonstrating rapid convergence as soon as mapped features are re-observed.

- **Middle-Case Scenario**: Under moderate operating conditions, with fewer landmarks and higher measurement noise, the overall errors are higher (up to ± 1.0 meters in translation), and temporary trajectory deviations occur.

  However, the filter demonstrated high resilience: the Mahalanobis distance gating logic remains sufficiently robust to allow the system to correct the drift and restore spatial consistency by performing a loop closure upon returning to already mapped areas.

- **Worst-Case Scenario**: In a highly sparse environment and with the highest level of noise, pose estimation errors become substantial, reaching up to 2.0 meters. The robot relies heavily on odometry, causing a visible drift in the estimated trajectory compared to the real one.

  Nevertheless, the Data Association logic continues to overlap almost perfectly with the ideal baseline system: the severe loss of accuracy is solely attributable to the

lack of environmental geometric constraints and not to failures in the association algorithm, which continues to avoid introducing false positives even under extreme uncertainty.

Bibliography

[1] The Unscented Kalman Filter, *Eric A. Wan and Rudolph van der Merwe*

[2] Unscented Kalman Filter Tutorial, *Gabriel A. Terejanu*

[3] *Positioning Systems 25-26: Lecture notes*, 2025-26

[4] Simultaneous Localization and Mapping: Part I, *Hugh Durrant-Whyte and Tim Bailey*

[5] Simultaneous Localization and Mapping: Part II, *Hugh Durrant-Whyte and Tim Bailey*

[6] SLAM for Dummies: A Tutorial Approach to Simultaneous Localization and Mapping, *Søren Riisgaard and Morten Rufus Blas*